# MECC: Single-GPU MoE Inference with Compiler-Guided Three-Tier Expert Caching

***Abstract*—Single-GPU mixture-of-experts (MoE) inference can stall on HDD reads when expert weights exceed the HBM and host memory assigned to a task. Managing HBM and host residency independently can discard experts that later tokens reuse. We present MECC (MoE Expert Cache Compiler), which couples profile-guided initial placement with joint HBM/CPU retention and dependency-aware weight delivery. The compiler assigns per-layer capacities; the runtime updates resident experts from unchanged router selections. In single-run FP16 Mixtral-8x7B measurements on one A100, MECC with uniform initialization delivers decode speedups of 1.77× over local streaming and 2.95× over fixed-CPU retention, with matching sample perplexity.**

***Index Terms*—Mixture of experts, single-GPU inference, expert offloading, three-tier caching, hardware/software co-design.**

## I. Introduction

Mixture-of-experts (MoE) models increase parameter capacity while activating only a few experts per token [1]–[4]. Deployment must still keep every expert addressable because a later token may select a different subset. This paper studies single-GPU inference when the expert set exceeds HBM and cannot be retained entirely in the host memory assigned to a task. The remaining weights reside on HDD. A selected expert must then be read into host memory and copied to the GPU before execution. Routing is resolved layer by layer, so the current gate does not reveal the next layer's expert demand. At batch one, a demand read can delay the next decoding step. Dense layers, KV cache, and staging buffers also compete with experts for HBM. The problem is to reduce these deliveries under fixed fast-tier allowances while preserving the model's original routing and weights. This setting matters for local inference and shared servers, where one task may receive only part of the machine's physical memory.

Existing systems address several parts of this problem. Multi-GPU expert parallelism distributes resident weights across accelerators [5]–[7]. Single-GPU offloading systems exploit activation traces, CPU execution, prefetching, or transfer pipelines [8]–[12]; SSD-LLaMA and edge FlashMoE further optimize disk delivery and replacement [13], [14]. These mechanisms make initial placement, admission, and retention consequential choices. A fixed host pool can lose an HBM victim that was not retained on the host. Independently updated caches can also spend slots on copies of the same expert. Preserving reuse across this boundary is a distinct decision from predicting the next expert or reducing the bytes transferred per miss. A remaining question is how a profile-derived initial state and online replacement should coordinate HBM and host residency when some experts must stay outside both tiers. Figure 1 isolates the runtime aspect with local controls on one A100. Separate HBM/CPU caches take 32.94 s per decode token, versus 7.38 s with joint retention under the same nominal slots and initialization. Their reported HDD-demand shares are 47.9% and 12.5%, respectively. These observations motivate preserving reuse across the tier boundary; they do not constitute a reproduction of the cited systems.

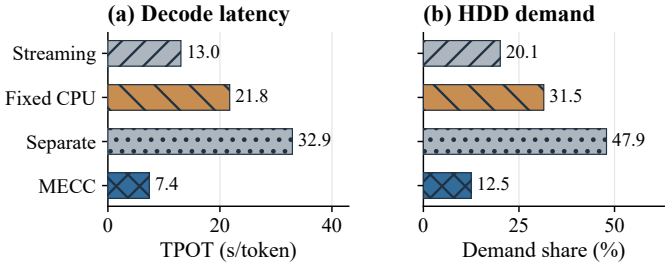



Fig. 1. Local retention controls on one A100 with Mixtral-8x7B. Separate denotes independently managed HBM/CPU caches. All policies use the same nominal slots and eight-prompt input. HDD-demand share includes all instrumented phases. One run per policy.

We present MECC, the MoE Expert Cache Compiler, to connect initial placement with runtime retention. An independent route profile supplies an expert-demand prior. The compiler ranks experts within each layer and allocates HBM and CPU slots under the task's allowances. Each plan retains the profiled layer/expert IDs, since equal indices in different layers refer to different weights. At runtime, the unchanged router provides actual demand. A CPU hit promotes the selected expert and retains the displaced HBM expert in CPU memory, preserving it for later tokens. Online recency changes the resident identities as the workload changes, while the compiled capacities remain fixed. Weight delivery orders reads, copies, and GPU consumers through readiness dependencies. These decisions address different parts of the same lifecycle: the profile selects an initial state, joint retention preserves reuse, and the delivery contract makes requested weights available before execution. The current A100 evaluation uses uniform initialization to examine runtime behavior independently of a profile-placement gain.

This paper makes three contributions.

(1) **Placement compilation:** we define a per-layer interface that maps route profiles and task allowances to ranked expert IDs and initial tier capacities, together with a marginal-coverage allocation heuristic.

(2) **Joint retention and delivery:** we specify promotion, victim demotion, and readiness dependencies across HBM, CPU pinned memory, and HDD while retaining the router and FP16 weights.

(3) **Single-GPU evaluation:** we implement an end-to-end harness and compare the runtime with streaming, fixed-CPU retention, independent caches, and local alternative policies on Mixtral-8x7B. With uniform initialization, MECC takes 7.38 s per output token, compared with 13.04 s for streaming and 21.75 s for fixed-CPU retention; the latter two controls report the same sample perplexity as MECC. The evaluation also examines source-tier demand and host-slot sensitivity.

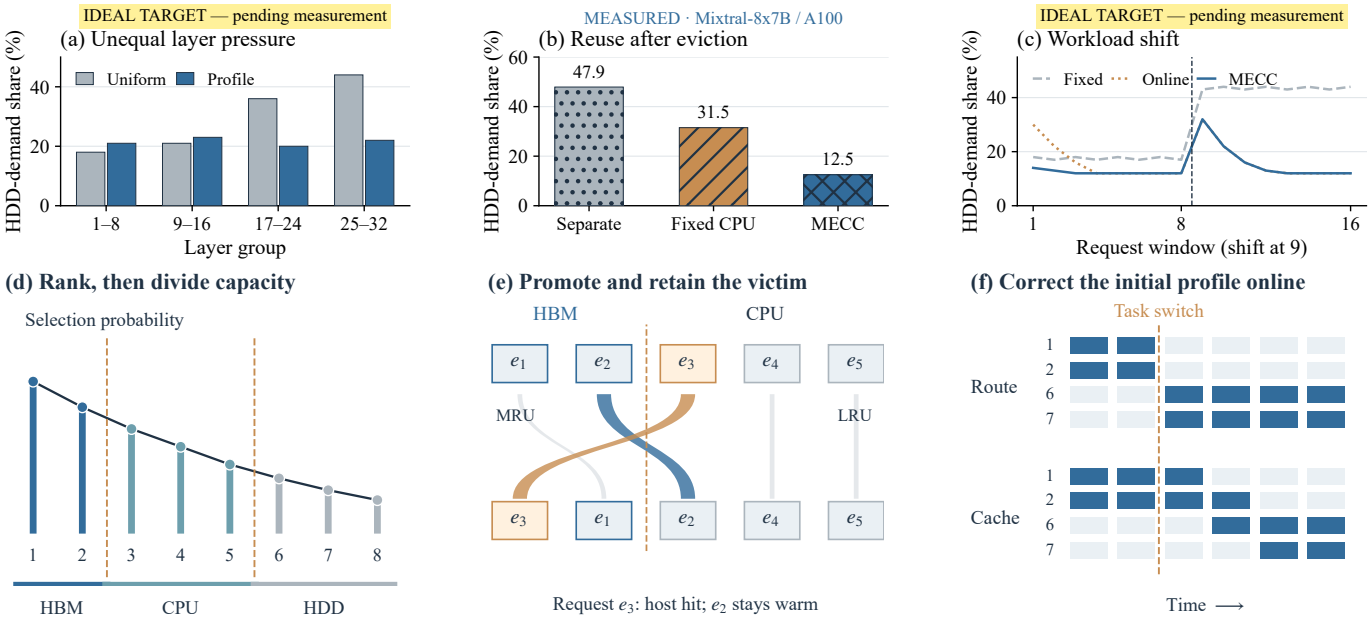



Fig. 2. Motivation evidence and matching insights. Panels (a,c) are ideal targets to replace; (b) is measured A100 demand across all phases. (d–f) are schematic mechanisms, adapted from the talk. Profiles guide initial coverage, joint retention preserves victims, and online updates follow changed routing.

## II. Background

An MoE layer routes each token to a small subset of its experts. Sparse execution reduces arithmetic, but every expert remains a possible future selection [1], [2], [15]. Mixtral and the DeepSeek family instantiate this separation between active computation and total weights [3], [4], [16]–[18]. Experts with the same index in different layers are different weight objects. A route profile must therefore retain both layer and expert identity.

Expert parallelism partitions weights over resident GPU memories and dispatches token activations to their owners [5]–[7], [19], [20]. On one GPU, the task also needs space for dense layers, activations, and the KV cache. KV management and request execution are complementary to expert residency [21], [22]. A host-resident expert needs an H2D copy; an expert absent from both fast tiers additionally needs a backing-store read. Heterogeneous inference systems expose related placement and scheduling choices [23]–[27].

We use immutable FP16 experts and three logical tiers: HBM, CPU pinned memory, and HDD. The task's allowance may be smaller than the server's physical RAM. Unique resident weights, host mirrors, staging buffers, and other allocations must be distinguished. The original gate determines demand, and a delivered expert can remain resident for later tokens. MECC separates the initial residency decision from the online transitions that respond to those later selections.

## III. Motivation and Insights

### A. Spend Capacity Where It Covers Demand

Equal slots per layer assume comparable cache pressure. A profile can reveal different ranked demand distributions even when each layer has the same expert count. Figure 2(a,d) connects this question to a ranked capacity cut: the compiler allocates the next slot by its marginal covered demand. E05: replace panel (a)'s ideal numbers with a same-runtime, same-byte placement sweep. The experiment must include empirical-profile and uniform controls; the current measured main runs use uniform initialization. Local routing consistency can also vary across models [28].

### B. An HBM Victim Can Still Be Useful

Figure 2(b,e) shows why two resident sets should preserve cross-tier reuse. With the same nominal slots and initialization, the measured independent-cache control reports 47.9% HDD demand, fixed CPU 31.5%, and MECC 12.5%. These counters span all instrumented phases. The insight is to promote a host hit while retaining the displaced HBM expert in CPU, rather than treating the eviction as the end of its useful lifetime. The source-tier and physical-byte measurements answer different questions and must be reported separately.

### C. Use the Profile as an Initial State

A profile represents past inputs. If the routed set changes, fixed residency can preserve experts that are no longer useful. Figure 2(c,f) pairs an expected miss transient with the corresponding residency update. Compilation fixes capacities and ranked IDs; online recency changes identities within those capacities. E07/E08: measure initial misses and the WikiText-2/GSM8K switch transient using distinct, hashed input sets. The experiment must retain the no-switch control and compare profile-plus-online with both fixed residency and online-only initialization.

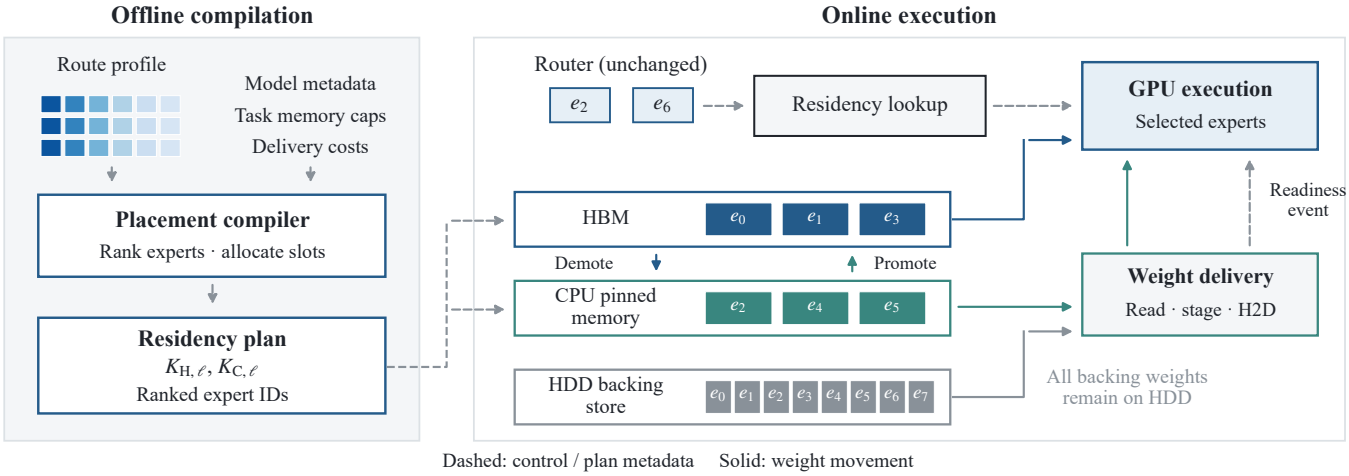



Fig. 3. MECC framework. The placement compiler emits per-layer capacities and ranked IDs. Online execution resolves unchanged router selections against HBM, CPU pinned memory, and HDD, then delivers weights before their GPU consumers. Dashed arrows carry control or plan metadata; solid arrows move weights.

Figure 3 separates offline compilation from online execution. An independent route profile, model metadata, task memory caps, and delivery costs define the compilation inputs. The output specifies layer capacities and initial expert IDs. Mirrors and staging are charged outside unique primary residency; HDD retains all backing weights.

The unchanged router supplies demand IDs. Residency lookup selects a source tier; the runtime promotes requested experts and demotes HBM victims. The delivery path enforces weight readiness before execution. Layer capacities stay fixed during a run, while expert identities and recency change. Planner controls therefore fix online rules; replacement controls fix quotas and initialization (Section VI-C).

## V. Method

### A. Compiler: Profile-Guided Placement

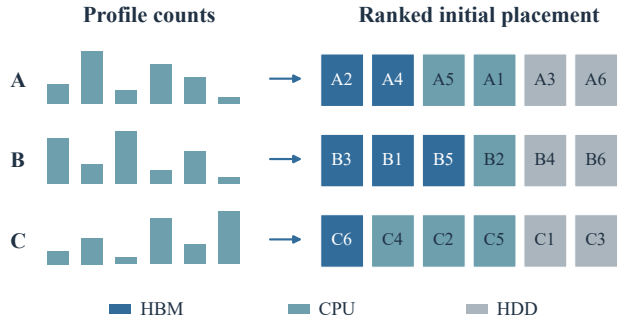



Fig. 4. Compiling layer-specific placement. A, B, and C denote different layers. The compiler keeps each layer's expert identities, ranks its profile counts, and allocates HBM and CPU slots from shared task allowances. Colors identify initial tiers; counts and capacities are schematic.

Figure 4 shows the compiler's input and emitted state. The compiler takes model metadata, memory budgets, delivery costs, and an independent route profile. The profile supplies request probability $p_e$ for each expert. For equal-size experts within a layer, a discrete Zipf model summarizes the ranked probabilities:

$$q_{\ell,r} = \frac{r^{-\alpha_\ell}}{H_{N_\ell}(\alpha_\ell)}, \qquad H_n(\alpha) = \sum_{r=1}^{n} r^{-\alpha}. \tag{1}$$

Here $q_{\ell,r}$ is conditional on an expert selection in layer $\ell$. Let $w_\ell$ be that layer's fraction of all selections. The global probability is $p_e = w_\ell q_{\ell,r}$, with $\sum_\ell w_\ell = 1$. For fixed top-$k$ routing, $w_\ell = k_\ell / \sum_j k_j$. Each rank retains its profiled expert ID. Empirical probabilities can replace the fitted Zipf distribution. Neither representation encodes request order.

Let $x_{e,t} \in \{0, 1\}$ indicate the primary tier of expert $e$. The tiers are HBM, CPU, and HDD, denoted by H, C, and S. Residency budgets $\bar{B}_t$ must exclude buffers and mirrors. The following static objective is a reference model, not a claim that the current planner solves the full problem:

$$\begin{aligned} \min_{x} \quad & \sum_e p_e \sum_t x_{e,t} T_t(s_e), \\ \text{s.t.} \quad & \sum_t x_{e,t} = 1, \quad \sum_e s_e x_{e,t} \le \bar{B}_t \quad (t \in \{\mathrm{H}, \mathrm{C}\}). \end{aligned} \tag{2}$$

The affine model $T_t(s) = a_t + s/b_t$ represents fixed overhead and effective bandwidth. Calibration uses the target machine. Equation 2 estimates static delivery cost under the profile. Runtime locality, migration, and overlap are evaluated separately.

For equal-size experts with fixed probabilities, exchanging a more probable expert in a slower tier with a less probable expert in a faster tier cannot increase static expected cost. This establishes ranked placement under independent capacities and ordered tier costs. The implementation below uses marginal coverage, rather than claiming to solve the general latency objective.

The multilayer implementation allocates slots by marginal coverage. Under the Zipf model, a slot covering rank $r$ adds

$$\Delta_\ell(r) = w_\ell q_{\ell,r}. \tag{3}$$

This is a coverage score, not the exact latency marginal of Equation 2. Tier-boundary movements can affect both HBM and CPU assignments. The heuristic allocates HBM slots first, then CPU slots from the remaining ranks. At each step it selects the largest next-rank score; ties use layer ID. Expert-ranking ties use expert ID. It stops at the tier's slot cap or exhaustion of experts. Per-layer minima, if enabled, are assigned first and an infeasible minimum allocation must be rejected. A direct scan costs $O(L(S_\mathrm{H} + S_\mathrm{C}))$ after ranking, where $S_t$ is a total slot cap. These rules define the compiler interface. Batched prefill additionally requires distinguishing token selections

from distinct expert fetches. The sequential heuristic does not solve the general static objective.

The plan records per-layer capacities, ranked expert IDs, initial tiers, and modeled hit fractions. A missing profile selects natural expert order. The measured runs below use this uniform/natural-order path, so they test the runtime retention and delivery stack rather than the profile compiler. Profile initialization cost and gains require separate matched runs.

### B. Runtime: Cross-Tier Retention

Each layer has ordered HBM and CPU residency maps. An HBM hit updates recency. A CPU hit removes the expert from CPU residency and promotes it to HBM as most recently used. An HDD miss reads the weights into staging before H2D transfer. When HBM is full, its least-recently used expert is demoted to CPU. CPU pressure evicts the least-recently used host resident. The HDD backing weights remain available.

Figure 5 starts with HBM $[e_1, e_2]$ and CPU $[e_3, e_4]$, each ordered from MRU to LRU. Requesting $e_3$ promotes it and demotes $e_2$. A later request for $e_2$ hits CPU. This example uses exclusive logical residency. An implementation can keep a CPU mirror to avoid device-to-host (D2H) transfer on eviction. The mirror consumes host memory and is charged separately.

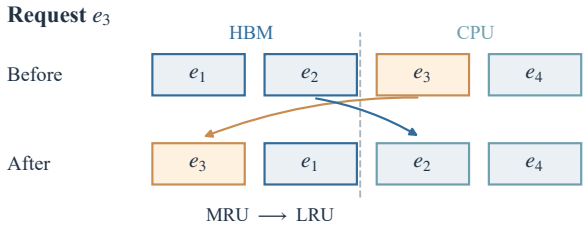



Fig. 5. One CPU-hit transition. Lists run from MRU to LRU. Requesting $e_3$ promotes it, demotes the HBM victim $e_2$, and preserves both experts in the combined fast-tier set. The dashed line is the capacity boundary.

For a serial demand stream within one layer, concatenate the HBM MRU-to-LRU stack with the CPU MRU-to-LRU stack. Exclusive promotion and victim demotion preserve the corresponding LRU stack of capacity $K_H + K_C$. This reference requires matching initialization, no prefetch or alternative admission, and completed transitions before the next request. It does not imply equivalence to a cache that reallocates capacity across layers, or dominance over static placement.

### C. Delivery: Dependency-Aware Overlap

A correct asynchronous implementation requires a readiness event after each pinned-host copy. The computation stream must wait before consuming the weight. The source buffer must remain live until copy completion, and the resident weight until its last consumer finishes. These are required lifetime rules, including across separate loading and dispatch iterators; the execution stream must enforce them with readiness events.

At batch one, routing reveals the selected IDs only after the current layer's gate. Copying one selected expert may overlap computation of another already-ready expert if kernel and aggregation order permit. This provides no knowledge of a later layer's route. In-flight weights must be protected from eviction. Replacement comparisons must disable predictive prefetch; the overlap control must hold admission, residency, and requests fixed. The available overlap and its exposed critical path remain measurements, rather than an assumed gain from a copy stream.

Figure 6 pairs HBM-resident $e_1$ with HDD-backed $e_3$ from the same gate. HDD read precedes H2D and readiness. Copy completion releases the pinned source; final-consumer completion permits GPU eviction. Useful independent computation determines the available overlap.

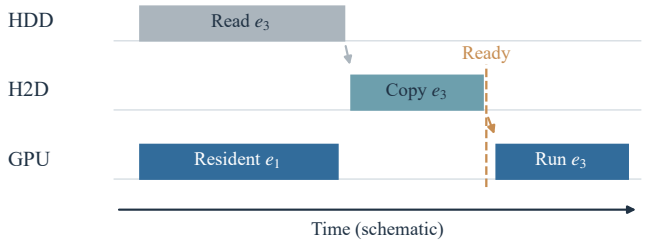



Fig. 6. Delivery dependencies within one gate's selected set. The GPU can execute resident $e_1$ while $e_3$ is read and copied. A readiness event orders the copied weight before its GPU consumer. Durations are schematic.

The design requires unchanged weights, selected experts, kernels, and aggregation order. Under these conditions, changing residency preserves the forward computation. The implementation also needs correct synchronization. The validation protocol compares synchronous and asynchronous paths on identical inputs using weight hashes, route selections, logits, tokens, and perplexity; the same protocol isolates transfer scheduling from changes in numerical execution.

### D. Implementation and Accounting

Policies share the FP16 loader and unchanged router. Logical tier occupancy is not physical memory. Host mirrors, staging, and temporary copies are charged separately from unique residents. Full CPU mirroring requires host bytes at least as large as the mirrored HBM experts. E14: replace nominal slot equivalence with measured physical-byte accounting.

The harness records initialization, warmup, prefill, decode, and PPL separately. TPOT and TTFT use their own timing windows. Demand-source counts and physical HDD/H2D/D2H bytes must share phase boundaries. E15: complete independent-resident reference checks before a general lossless claim.

## VI. Evaluation

### A. Testbed and Evidence Map

Measured runs use FP16 Mixtral-8x7B-Instruct-v0.1 on one A100 80 GB PCIe. The cold store is on a RAID logical device reporting rotational media; physical drives remain uninventoried. Main runs use four HBM and three host slots per layer, uniform initialization, eight WikiText-2 prompts, 64 input and 64 output tokens, and one run per policy. Perplexity uses 1024 WikiText-2 tokens. The 50/40 GB allowances are planner parameters; mirrors and buffers still need a physical-byte audit.

Yellow plots and text are experiment-completion targets, not measurements. Their arrays come from the existing design-target record. Freeze model revisions, input hashes, precision, total charged bytes, and timing windows before replacing each target. The experiments address full-system benefit, individual mechanisms, workload change, resource tradeoffs, and numerical equivalence.

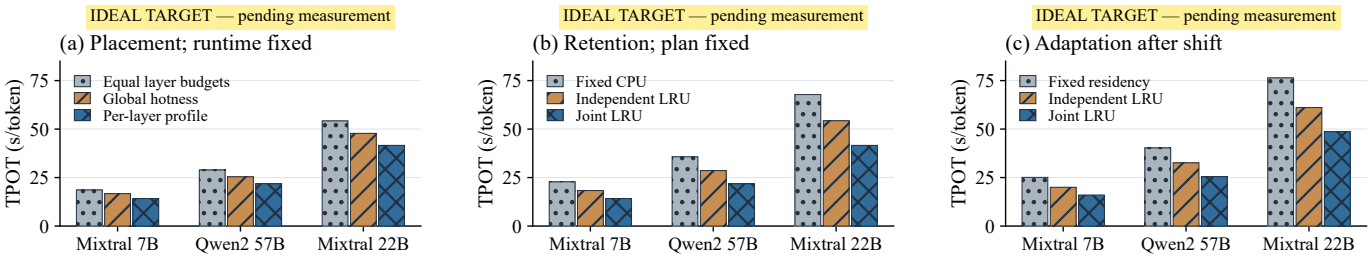



Fig. 7. TO MEASURE: ideal component targets on three models. (a) Placement with runtime fixed. (b) Retention with initialization fixed. (c) Adaptation after a task switch. Separate protocols must be frozen for the three controls; these bars are not A100 outcomes.

### B. Measured Runtime and Quality

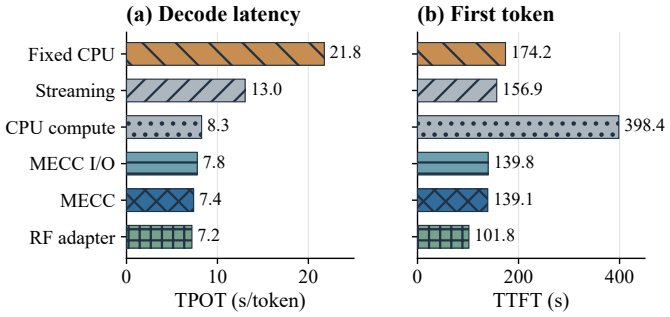



Fig. 8. Measured single-A100 runtime: one run per local policy, eight WikiText-2 prompts, 64 output tokens. RF and CPU-compute adapters are local mechanisms.

Figure 8 reports MECC at 7.38 s/token, streaming at 13.04, and fixed CPU at 21.75: TPOT ratios of 1.77× and 2.95×. The RF adapter takes 7.19 s, MECC I/O 7.78, and CPU compute 8.26. RF is faster than MECC in this run. MECC's TTFT is 139.1 s, versus 156.9 for streaming and 174.2 for fixed CPU. MECC, streaming, fixed CPU, RF, and MECC I/O report PPL 3.427; CPU compute reports 3.432. A short-window route/logit/token diagnostic supports local correctness but does not replace an independent resident reference.

### C. Mechanism Attribution

The matched local independent-cache control takes 32.94 s/token. Figure 7 separates three mechanisms in the target component harness. For Mixtral-8x7B, the placement target moves from 18.6 to 14.2 s/token, joint retention from 18.3 to 14.2, and post-switch adaptation from 25.1 to 16.0 relative to fixed residency. The three controls hold runtime, initial plan, and switch protocol fixed, respectively. This separates better initial coverage from preservation of later reuse. E04–E08: replace targets and add leave-one-out controls.

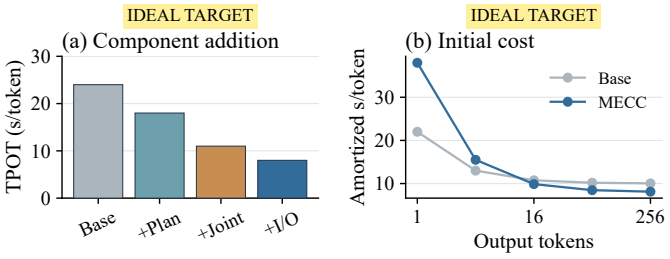



Fig. 9. TO MEASURE: ideal ablation and startup targets. The right panel evaluates startup/output-token plus decode cost from authored inputs; it is not a measured latency trace.

Figure 9 tests component addition and startup amortization. The authored TPOT sequence is 24, 18, 11, and 8 s/token. The last I/O step retains a 20-percent target HDD-demand share, separating delivery scheduling from miss reduction. With the plotted startup/decode costs, the target amortization curves cross at 9 output tokens. This check exposes the service horizon over which an initial investment pays back. E04/E13: replace targets with phase timings and addition/removal runs.

### D. Model and Deployment Coverage

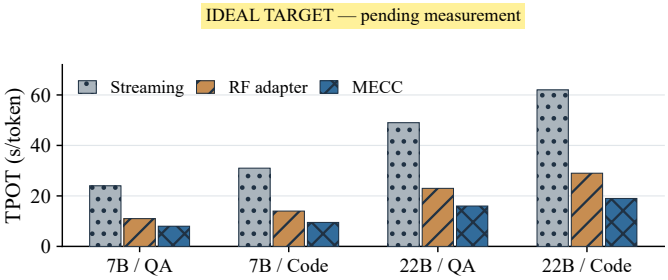



Fig. 10. TO MEASURE: ideal model/workload targets. Mixtral-8x7B and 8x22B with QA/code inputs. Values specify planned comparisons, not measured benchmarks. The RF policy is a local adapter.

Figure 10 expands the present one-model workload. The MECC target ranges from 8 to 19 s/token, versus 24 to 62 for streaming and 11 to 29 for the RF adapter. QA and code exercise different expert-demand patterns; increasing model size also changes the expert pool and transfer size. The comparison therefore pairs every policy within each model/workload point. E01/E02: replace targets under frozen inputs, precision, and byte allowances.

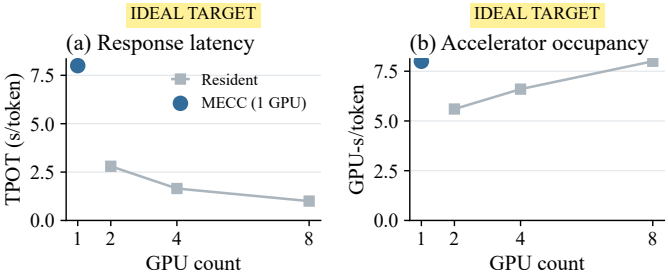



Fig. 11. TO MEASURE: ideal one-GPU/resident-reference tradeoff. MECC has only a one-GPU point. Resident points use 2/4/8 GPUs; occupancy is GPU count times TPOT. These values are not published FlashMoE results.

Figure 11 separates GPU provisioning from response latency. The endpoint target is one GPU at 8 s/token versus eight GPUs at 1 s/token; both correspond to 8 GPU-s/token. The two-GPU resident target is 5.6 GPU-s/token, which makes the smallest feasible resident configuration a necessary comparison. Occupancy is reported alongside latency, rather than substituted for energy. E03/E14: replace all resident-reference targets.

### E. Capacity and Implementation Checks

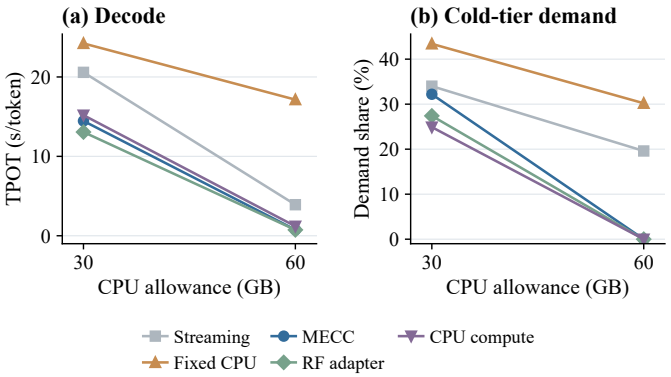



Fig. 12. Measured 30/60 GB nominal host-allowance comparison. Four WikiText-2 prompts and 16 output tokens, one run per point. Demand share covers all instrumented phases.

Figure 12 reports MECC at 14.46 and 0.76 s/token, with 32.2% and zero HDD demand. Four HBM plus four host slots at 60 GB cover all eight experts per layer; zero cold demand is a capacity transition. RF takes 13.06 and 0.74 s/token. These points use a shorter protocol than the main run.

IDEAL TARGET — pending measurement

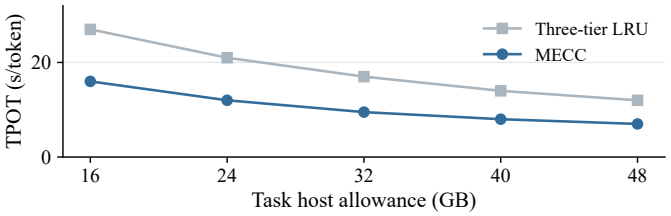



Fig. 13. TO MEASURE: ideal dense-quota sweep. The target experiment uses a separately frozen protocol; it is not joined to the measured 30/60 GB curve.

Figure 13 resolves intermediate capacity points. Its authored MECC target changes from 16 to 7 s/token over 16–48 GB, compared with 27 to 12 for three-tier LRU. The purpose is to separate pressure within the cold-tier regime from the full-coverage threshold. E10: replace with one matched, physically charged quota sweep.

E14/E15: fill peak physical HBM/host bytes, mirrors, pinned staging, phase-specific traffic, and independent-reference route/logit/token checks. The current sample PPL equality supports only that sample. E09/E11/E12: add profile-size sensitivity, no-switch/shift controls, and batch/input/output-length sensitivity if the corresponding claims remain in the paper.

TABLE I
NUMERICAL AND RESOURCE CHECKS. SAMPLE PPL VALUES ARE MEASURED; HIGHLIGHTED ENTRIES REQUIRE INDEPENDENT VALIDATION OR MEASUREMENT.

| Check | Current value / target |
|---|---|
| MECC / fixed-CPU sample PPL | 3.427 / 3.427 |
| CPU-compute sample PPL | 3.432 |
| Resident-reference route match | Target: exact |
| Resident-reference logit error | Target: zero |
| Physical HBM / host peak bytes | TO MEASURE |
| Phase-specific HDD/H2D bytes | TO MEASURE |

Table I closes the evaluation contract: performance comparisons preserve the routed computation and precision, and charge the actual resident and temporary allocations. A nominal slot budget alone cannot establish physical-memory equivalence.

## VII. Related Work

Expert offloading starts from the distinction between sparse execution and full weight capacity [8], [29]. Fiddler and DAOP choose CPU/GPU execution or movement; MoE-Lightning pipelines heterogeneous work; MoE-Infinity uses activation traces [9]–[11], [30]. ProMoE, Pre-gated MoE, Fate, and ExpertFlow provide lookahead, proactive caching, or token scheduling [12], [31]–[33]. MECC's demand path instead consumes the original gate output and coordinates initial placement with online residency.

HOBBIT and Specter use precision in expert loading; MoEpic divides experts into segments; SliceMoE caches precision slices; Mira combines staging with quantization [34]–[38]. ZipMoE uses compression and affinity scheduling [39]. SSD-LLaMA, edge FlashMoE, and MoE-CORE address storage delivery, replacement, or coordinated residency [13], [14], [40]. MECC studies the interaction between an independent placement prior and online whole-expert retention across the two fast tiers under HDD-backed demand.

Granularity changes the meaning of a cache hit. A segment cache can avoid part of an expert transfer while still fetching the remaining weights. A precision-slice cache can change both resident bytes and numerical execution. A whole-expert cache instead changes the source of unchanged weights. These comparisons therefore report representation and quality alongside latency. Physical traffic per miss and the number of misses are separate quantities: compression or splitting can reduce the former, while retention changes the latter.

TABLE II
REPRESENTATIVE CONTROL GRANULARITY AND PRIMARY DECISION.

| System | Unit | Primary decision |
|---|---|---|
| Fiddler [10] | Expert | CPU/GPU execution |
| ProMoE [12] | Expert | Proactive caching |
| MoEpic [36] | Segment | Split ratio and allocation |
| HOBBIT [34] | Expert precision | Precision-aware loading |
| Bevy [41] | Request | Runahead and batching |
| MECC | FP16 expert | Initial placement and joint retention |

Klotski and Bevy expose independent work across batches or requests [41], [42]. OD-MoE distributes on-demand loading over edge nodes, while HeteGen studies heterogeneous dense-model inference [43], [44]. Their execution units differ from batch-one per-gate expert delivery. The full-system, mechanism, and resource experiments above distinguish these comparisons.

Cross-request scheduling also changes the available overlap. A ready request can advance while another waits, providing work beyond the current gate's selected experts. In a single-request stream, MECC's delivery dependencies only expose already-ready expert computation. This distinction explains why cache replacement, route lookahead, and request scheduling require separate controls even when all three improve an end-to-end latency metric.

## VIII. Conclusion

MECC connects a profile-derived initial state with online cross-tier retention and weight delivery. Current single-A100 measurements support the local runtime comparison. The yellow experiment panels define the remaining evidence for placement, adaptation, model coverage, startup, and resident multi-GPU tradeoffs. They must be replaced before those outcomes enter a final performance claim.

## References


[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[2] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[3] A. Q. Jiang, A. Sablayrolles, A. Roux *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[4] D. Dai, C. Deng, C. Zhao *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," in *Proc. ACL*, 2024, pp. 1280–1297.

[5] S. Rajbhandari, C. Li, Z. Yao *et al.*, "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 18 332–18 346.

[6] C. Hwang, W. Cui, Y. Xiong *et al.*, "Tutel: Adaptive mixture-of-experts at scale," in *Proc. MLSys*, 2023.

[7] O. J. Aimuyo, B. Oh, and R. Singh, "FlashMoE: Fast distributed MoE in a single kernel," *arXiv preprint arXiv:2506.04667v3*, 2025.

[8] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.

[9] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, "MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache," *arXiv preprint arXiv:2401.14361v3*, 2025.

[10] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci, "Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models," in *International Conference on Learning Representations*, 2025.

[11] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, "MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

[12] X. Song, Z. Zhong, R. Chen, and H. Chen, "ProMoE: Fast MoE-based LLM serving using proactive caching," *arXiv preprint arXiv:2410.22134*, 2024.

[13] F. Liang, Y. Shen, J. Hu, J. Xu, H. Gao, M. Xu, and Z. Meng, "SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC," *arXiv preprint arXiv:2609.18110*, 2026.

[14] B. Kim, J. Lee, D. Han, H.-J. Yoo, and S. Kim, "FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices," *arXiv preprint arXiv:2601.17063v2*, 2026.

[15] D. Lepikhin, H. Lee, Y. Xu *et al.*, "GShard: Scaling giant models with conditional computation and automatic sharding," in *Proc. ICLR*, 2021.

[16] DeepSeek-AI, A. Liu, B. Feng *et al.*, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[17] ——, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[18] Mistral AI, "Cheaper, better, faster, stronger," Official Mixtral 8x22B release, 2024. [Online]. Available: https://mistral.ai/news/mixtral-8x22b

[19] T. Gale, D. Narayanan, C. Young, and M. Zaharia, "MegaBlocks: Efficient sparse training with mixture-of-experts," in *Proc. MLSys*, 2023.

[20] Z. Doucet, R. Sharma, M. de Vos, R. Pires, A.-M. Kermarrec, and O. Balmau, "HarMoEny: Efficient multi-GPU inference of MoE models," *arXiv preprint arXiv:2506.12417v2*, 2025.

[21] W. Kwon, Z. Li, S. Zhuang *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[22] L. Zheng, L. Yin, Z. Xie *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[23] Y. Sheng, L. Zheng, B. Yuan *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 31 094–31 116.

[24] R. Y. Aminabadi, S. Rajbhandari *et al.*, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. SC*, 2022.

[25] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning," *arXiv preprint arXiv:2104.07857*, 2021.

[26] K. Alizadeh, S. I. Mirzadeh, D. Belenko *et al.*, "LLM in a flash: Efficient large language model inference with limited memory," in *Proc. ACL*, 2024, pp. 12 562–12 584.

[27] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proc. SOSP*, 2024.

[28] J. Liang, S. Wang, M. Tian *et al.*, "Not all models suit expert offloading: On local routing consistency of mixture-of-expert models," *arXiv preprint arXiv:2505.16056v4*, 2026.

[29] R. Kong, Y. Li, Q. Feng, W. Wang, X. Ye, Y. Ouyang, L. Kong, and Y. Liu, "SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget," *arXiv preprint arXiv:2308.15030*, 2023.

[30] Y. Zhang, S. Aggarwal, and T. Mitra, "DAOP: Data-aware offloading and predictive pre-calculation for efficient MoE inference," *arXiv preprint arXiv:2501.10375*, 2025.

[31] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, "Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *Proc. ISCA*, 2024.

[32] Z. Fang, Z. Hong, Y. Huang *et al.*, "Fate: Fast edge inference of mixture-of-experts models via cross-layer gate," *arXiv preprint arXiv:2502.12224v2*, 2025.

[33] X. He, S. Zhang, K. Tang *et al.*, "ExpertFlow: Efficient mixture-of-experts inference via predictive expert caching and token scheduling," *arXiv preprint arXiv:2410.17954v2*, 2026.

[34] P. Tang, J. Liu, X. Hou *et al.*, "HOBBIT: A mixed precision expert offloading system for fast MoE inference," *arXiv preprint arXiv:2411.01433v2*, 2024.

[35] W. Wang, X. Hou, X. Xia *et al.*, "Specter: Fast MoE inference via quantization-enabled multi-step speculative offloading," *arXiv preprint arXiv:2511.14102v3*, 2026.

[36] J. Yan, J. Liu, H. Xu, and L. Huang, "Accelerating mixture-of-expert inference with adaptive expert split mechanism," *arXiv preprint arXiv:2509.08342*, 2025.

[37] Y. Choi, S. Kim, J. Oh, G. Park, B. Kim, and H.-J. Yoo, "SliceMoE: Bit-sliced expert caching under miss-rate constraints for efficient MoE inference," *arXiv preprint arXiv:2512.12990*, 2025.

[38] S. Yadav and B. Asgari, "Mira: Memory-efficient MoE inference using adaptive caching and predictive expert staging," *arXiv preprint arXiv:2609.38090*, 2026.

[39] Y. Yang, Y. Zhao, P. Yang, S. Wang, and Z.-H. Zhou, "ZipMoE: Efficient on-device MoE serving via lossless compression and cache-affinity scheduling," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 147 261–147 284.

[40] K. Yang, Y. Gao, X. Li *et al.*, "MoE-CORE: Coordinated expert offloading and residency for memory-constrained MoE inference," *arXiv preprint arXiv:2610.01950*, 2026.

[41] H. Yu, I. Gim, K. Lee, and L. Zhong, "Accelerating offloading-based MoE inference by interleaving concurrent requests," *arXiv preprint arXiv:2611.04794*, 2026.

[42] Z. Fang, Y. Huang, Z. Hong, Y. Lyu, W. Chen, Y. Yu, F. Yu, and Z. Zheng, "Klotski: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline," in *Proc. ASPLOS*, 2025.

[43] L. Wang, Y. Du, Y. Pan, S. C. Liew, J. Liu, and K. Chen, "OD-MoE: On-demand expert loading for cacheless edge-distributed MoE inference," *arXiv preprint arXiv:2512.03927*, 2025.

[44] X. Zhao, B. Jia, H. Zhou, Z. Liu, S. Cheng, and Y. You, "HeteGen: Heterogeneous parallel inference for large language models on resource-constrained devices," *arXiv preprint arXiv:2403.01164*, 2024.